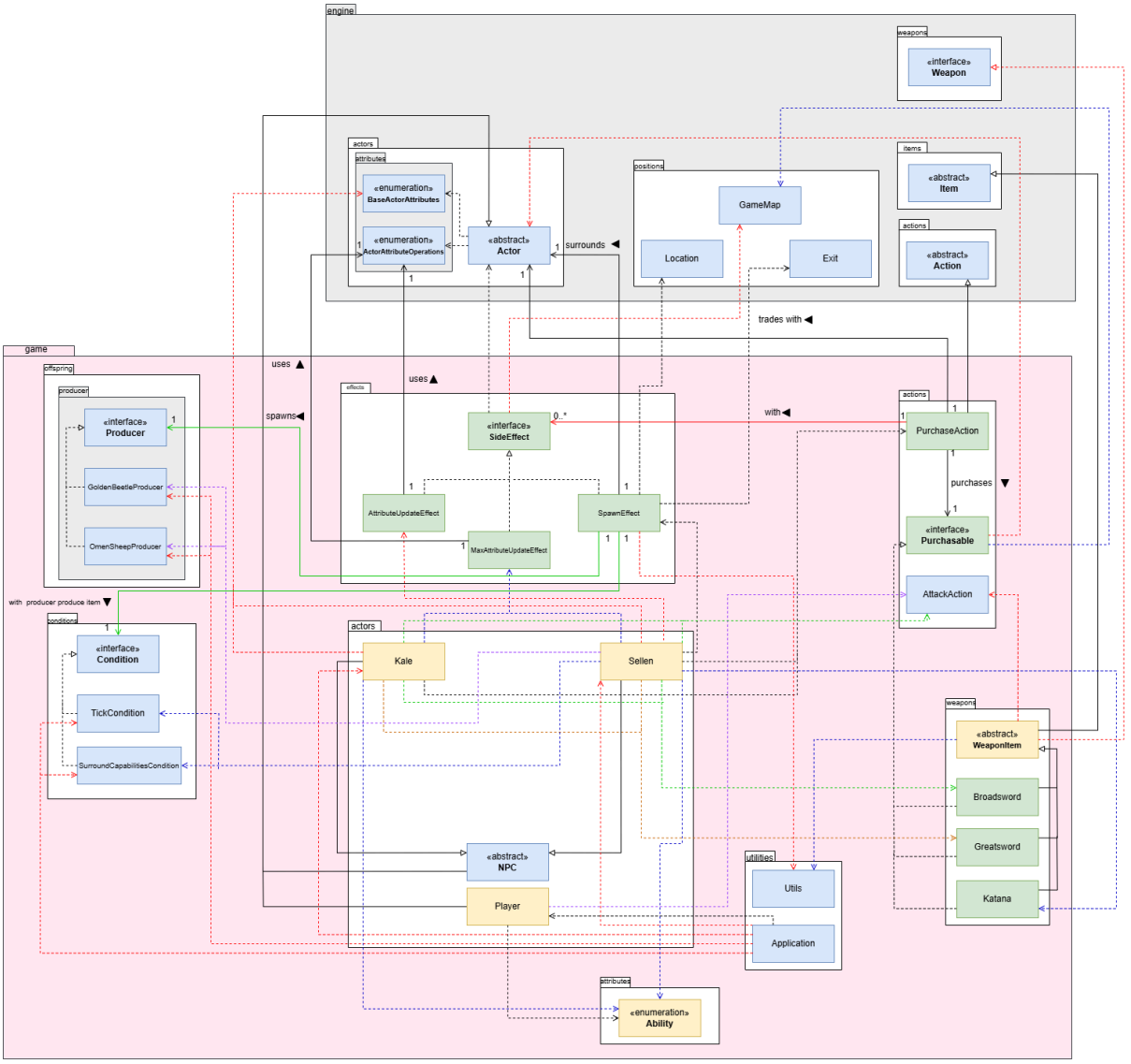
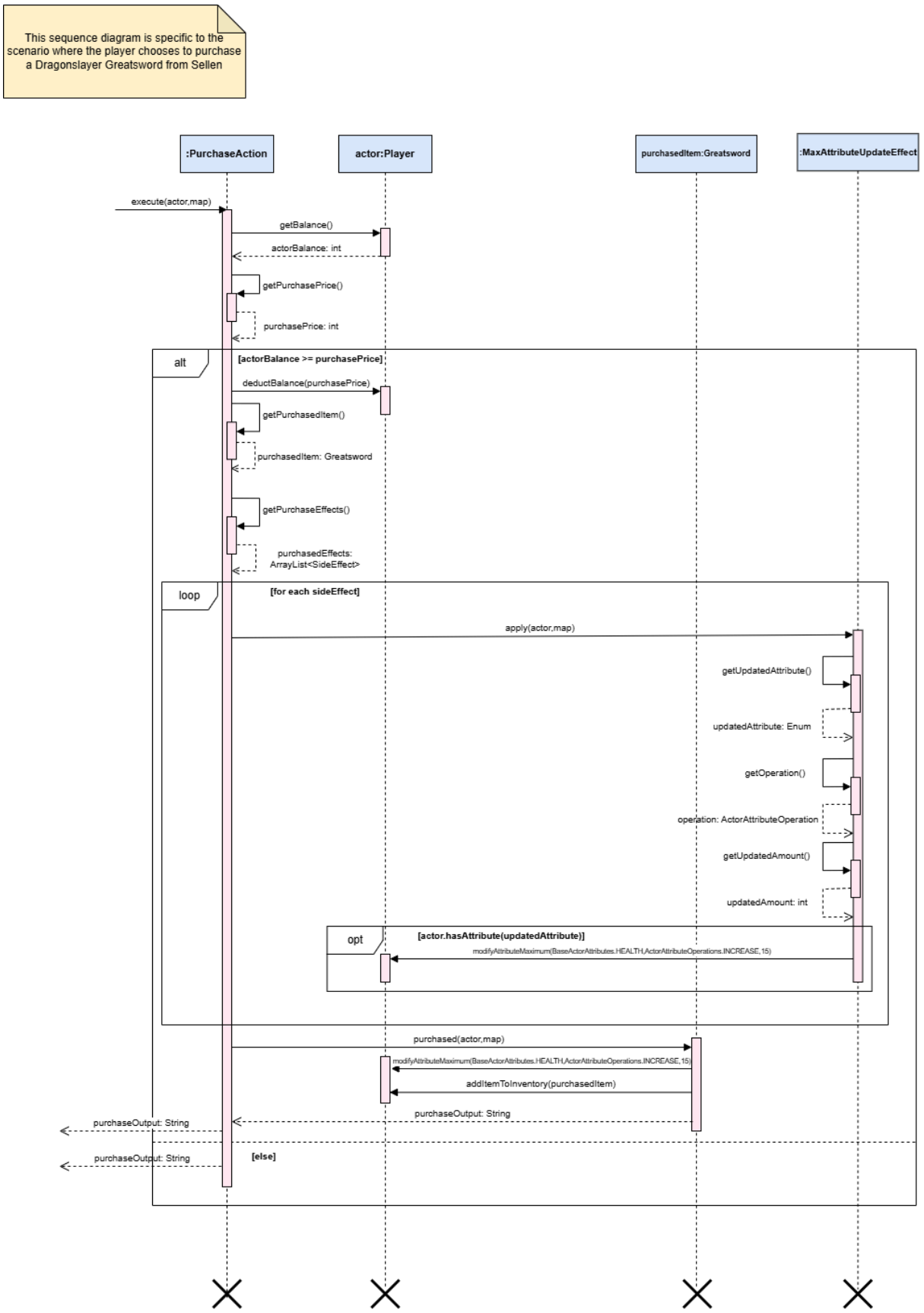
Design Documentation

Req 4 - The Wayward Seller of Wares

UML diagram:



Sequence diagram:



Design Rationale:

A. Implementation of Broadsword, Dragonslayer Greatsword, Katana

Design 1: Specific weapon items are created by instantiating the WeaponItem concrete class object with specific attributes' value.

Pros	Cons
Simple and less classes needed	Limited flexibility for specific weapon items to define own implementation logic
	Violating DRY when defining same parameter values for many instantiated WeaponItem objects
	Introduce connascence of values when many instantiated WeaponItem object must have same literal value

Design 2: Refactor WeaponItem to abstract class, creating concrete classes Broadsword, Greatsword and Katana by extending the abstract class.

Pros	Cons
Introduce flexibility for specific weapon items to define own implementation logic	Tighter coupling for deeper inheritance level
Minimize repeated code because WeaponItem abstract groups all weapon items' shared attributes and methods	
Reducing connascence of values by making the weapon items' constant attribute value as static final attribute	

<u>B. Implementation of purchase side effects that may be repeated</u>

Design1: Create an Effect enum class, and a SideEffect class, adding new enum values UPDATE_ATTRIBUTE, UPDATE_MAX_ATTRIBUTE, SPAWN_ACTOR, SideEffect class has attribute Effect enum type, and an apply() with switch cases to check the effect to be applied.

Pros	Cons
Less classes needed	Limited flexibility for side effects involve complex implementation logic
	Violate Open-Closed principle to modify SideEffect class's switch case implementation when new side effects are introduced.

Design 2: Create a SideEffect interface defining the methods that should be implemented by specific side effects. Create AttributeUpdateEffect, MaxAttributeUpdateEffect and SpawnEffect classes to implement the interface.

Pros	**Cons**
Modular code, the side effect classes can be reused	May introduce unnecessary complexity if some side effects only involve simple implementation logic.
Using interface allows all side effects be used interchangeably	
Improve extensibility when more side effects are introduced in future.	

C. Different purchasable entities can be purchased from different merchants with different prices, side effects

A PurchaseAction is created by extending Action abstract class given in the engine to manage the flow of purchasing an item, with attribute types:

Design 1: PurchaseOption and Actor(merchant). PurchaseOption is a class with attribute types int (price), Item (purchasedItem) and ArrayList<SideEffect> to act as a data driven class storing the purchase options' details.

Pros	**Cons**
Introduce encapsulation by encapsulating the purchase option's details	Unnecessary complexity when the PurchaseOption class's attributes can be PurchaseAction attributes.
Improve extensibility if more attributes are introduced to purchase options.	

Design 2: Actor(merchant), Purchasable, int(price) and ArrayList<SideEffect>, Purchasable is an interface representing purchasable items, Broadsword, Greatsword and Katana classes implement it.

Pros	**Cons**
Purchasable provides a better abstraction layer representing purchasable entities, because not all Items are purchasable, and provide scalability for entities other than item be purchasable.	Less flexibility if the purchase option logic needs to be reused in other places than PurchaseAction.
Simple structure	

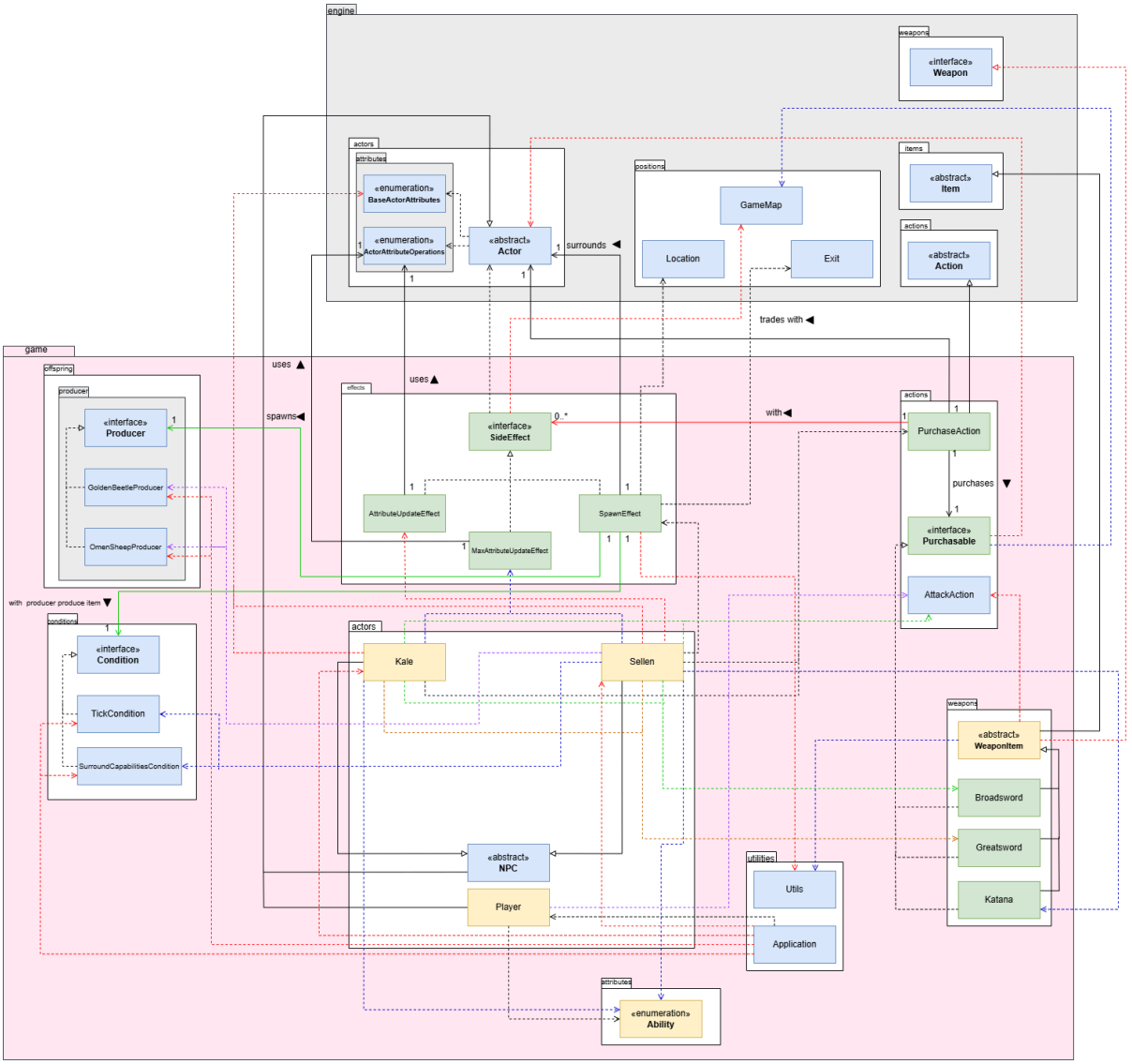
D. Storing and adding the purchase actions offered by merchants

Design 1: Add an ArrayList<PurchaseOption> type attribute and method addPurchaseOption(PurchaseOption) to merchant classes, then add the purchase options to them in Application class, allowableActions() loops the ArrayList<PurchaseOption> to create PurchaseAction with different PurchaseOption.

Pros	**Cons**
Provide flexibility to add new purchase options offered by merchants at the run time	Application takes additional responsibility to know purchase options offered by the merchants, violating Single Responsibility Principle.

Design 2: Add a loadPurchaseActions() in merchant classes to return the purchase actions offered by the merchant. Calling this method inside the allowableActions().

Pros	**Cons**
The merchant classes take their own responsibility to know their available PurchaseActions.	Less flexibility to add new purchase options offered by merchants outside the merchant classes
Separate purchase actions loading from allowableActions() minimizes the complexity of allowableActions()	



The diagram above shows the final design of requirement 4, which utilizes Design 2 (part A,B,C,D) mentioned above.

The new PurchaseAction extends the Action abstract class to introduce code reuse to reduce repeated code (DRY), because PurchaseAction can inherit its methods and attributes from the Action abstract class given in the engine which provides the actions' shared methods and attributes. Beside, the refactored WeaponItem abstract class is extended by specific weapon item concrete classes, Broadsword, Greatsword and Katana to reduce DRY because all weapon items' shared attributes and methods are grouped. Although this design creates more classes and introduces tighter coupling because of inheritance, it provides the flexibility for the specific weapon item classes to define their own implementation. Also, these weapon items have different constant attribute values, creating specific concrete classes minimizes connascence of values, no repeated attributes value need to be written multiple times whenever a specific weapon item be created. The constant attributes' values become static final attributes.

The PurchaseAction is also created to segregate the new responsibility of managing the flow of purchasing items, so the existing classes no need to take a new responsibility (Single Responsibility Principle). This class has association with Actor, SideEffect and Purchasable, because using these classes type attributes to adhere to the Open-Closed Principle, when more merchants, side effects and purchasable entities are introduced in future, no modification is needed in the PurchaseAction class.

The SideEffect interface is created and implemented by specific side effect classes. Although this design builds more classes, its advantages outweigh its disadvantages. Creating these side effect classes introduce modular code, so these side effect implementations can be reused in many places.

Besides, interfaces are used to provide abstraction layers for side effects and purchasable entities. These designs adhere to the Dependency Inversion Principle because other classes do not need to depend directly on specific side effects and purchasable entities concrete classes, other classes (e.g. PurchaseAction) can use any classes that implement the interfaces without knowing the specific class type. Thus, all classes implementing the interfaces can also be used interchangeably without impacting the usage in classes interacting with them (Liskov Substitution Principle).